GCB6206: Reinforcement Learning

Week 11

Youngwoon Lee

Plan for today

- Offline RL
- TD3+BC
- Conservative Q-Learning (CQL)
- Filtered BC
- Advantage-Weighted Regression (AWR)
- Implicit Q-Learning (IQL)

Offline RL learns from given static data

- Offline RL
  - Offline dataset is given
  - Train policy on offline data

- Online RL (on-policy or off-policy):
  - Collect data
  - Update policy on latest data or data so far

Offline RL learns from given static data

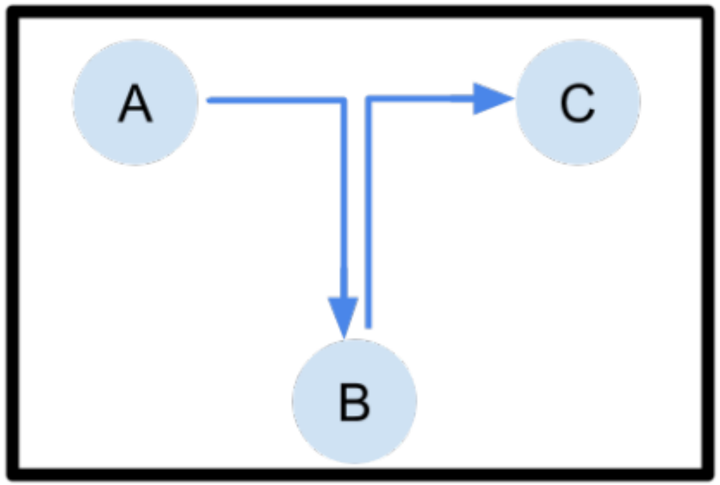
- Offline RL
  - Offline dataset is given
  - Train policy on offline data
- Offline dataset $\mathscr{D} : \{(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)\}$
  - Data collected from some unknown policy π_β, called *behavior policy*
  - $\mathbf{s} \sim \pi_\beta(\tau)$
  - $\mathbf{a} \sim \pi_\beta(\cdot \mid \mathbf{s})$
  - $\mathbf{s}' \sim p(\cdot \mid \mathbf{s}, \mathbf{a})$
  - $r = r(\mathbf{s}, \mathbf{a})$

π_β may be a mixture of policies

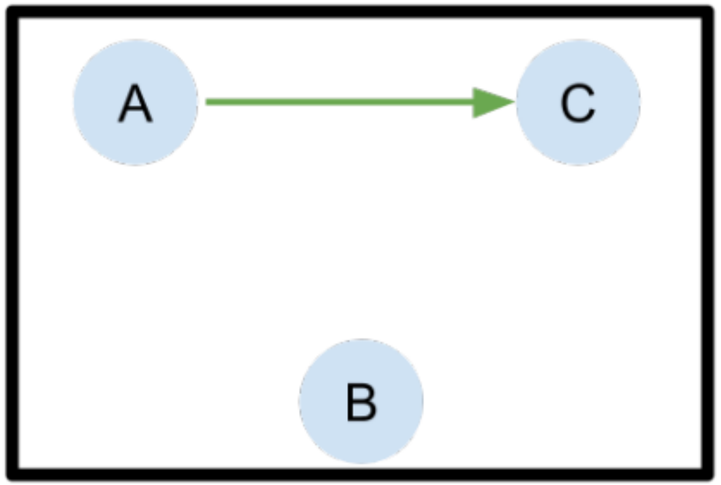
- Offline RL objective is the same: $\max_\theta \Sigma_t \mathbb{E}_{\tau \sim \pi_\theta(\tau)} \left[r(\mathbf{s}_t, \mathbf{a}_t) \right]$

How is offline RL different from BC?

Training data (A-B, B-C)



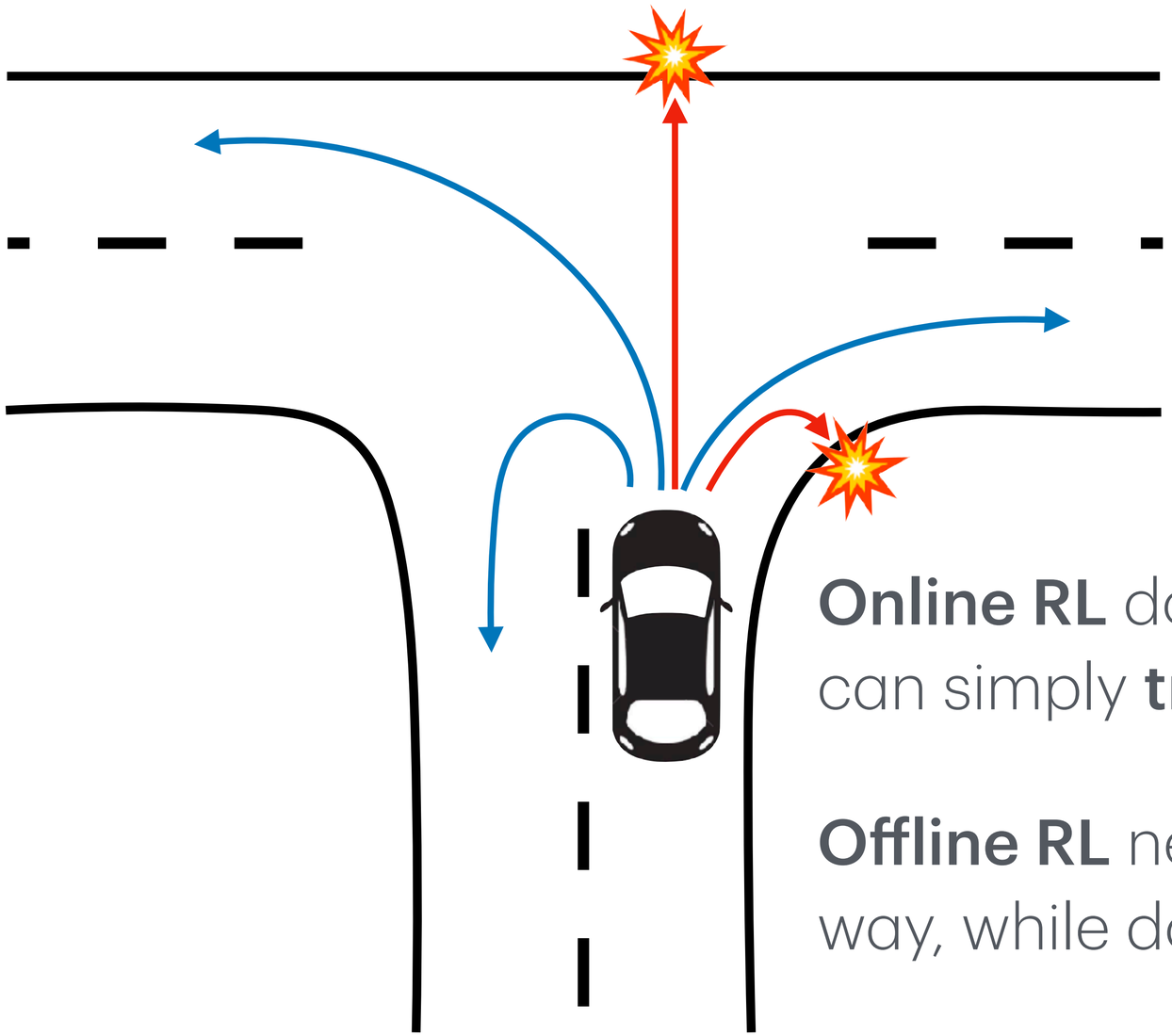
Find optimal path (A-C)



Offline RL can “stitch” trajectories

Offline RL can leverage bad data

Why is offline RL hard?



Training data

What the policy wants to try
→ Is this good or bad?

We never know since it's offline!

Online RL doesn't have to handle this, because they can simply **try** this action and see what happens

Offline RL needs to handle unseen actions in a safe way, while doing better than data

Levine et al. "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems" arXiv 2020

Why, or when, might offline RL be useful?

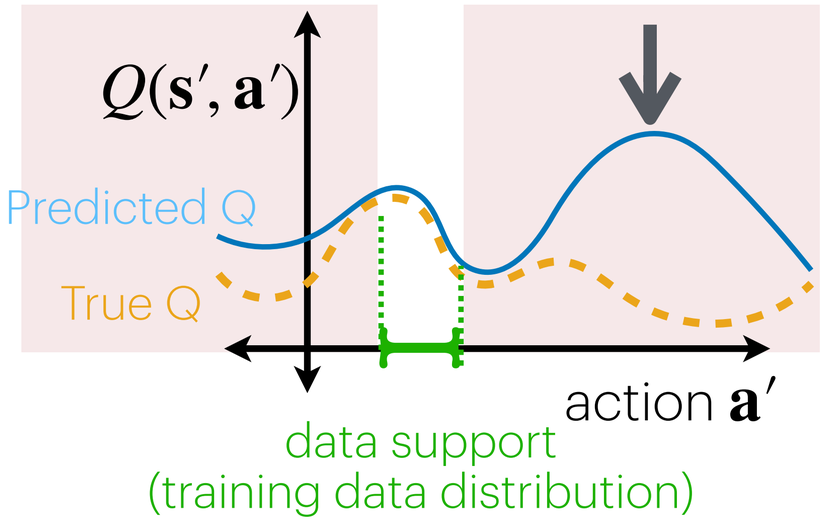
- Online policy data collection can be risky, unsafe, or expensive
  - Healthcare, robotics, autonomous driving, etc
- Reuse previously collected data, rather than recollecting
  - Data from previous experiments
  - Data from other projects
  - Data from other institutions
- Leverage datasets collected by humans or existing systems (not by the current policy)

Can we simply use off-policy algorithms?

Can we simply use off-policy algorithms?

This term is problematic again!

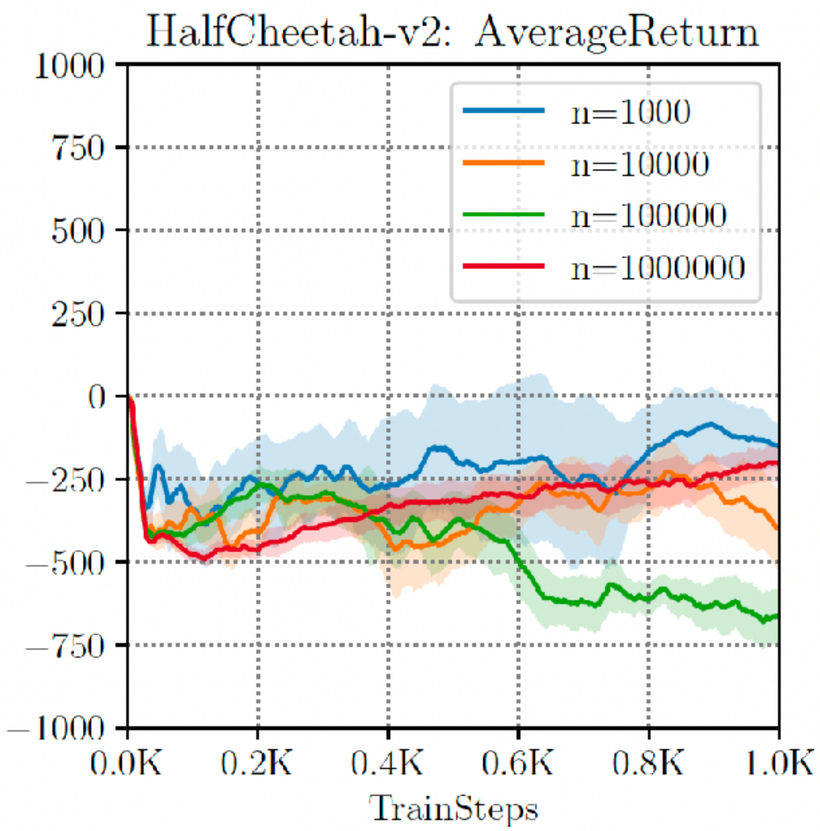
- In Q-learning, target value $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
- What happens if you optimize this using a **static dataset**?
- What happens when evaluating Q on actions $\mathbf{a}'$ not in the dataset?



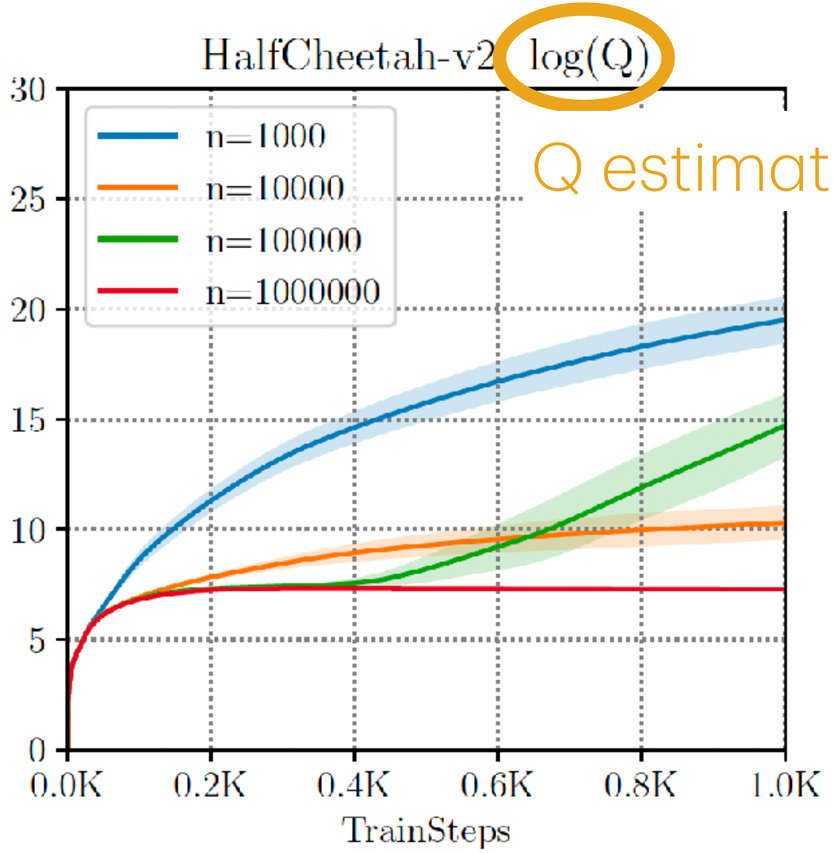
- Q-function is unreliable on out-of-distribution (OOD) actions
- $\max$ will seek out actions where Q-function is over-optimistic
- After values propagate, Q-value will become substantially overestimated

Can we simply use off-policy algorithms?

- Overestimation of Q-value when train SAC with a *static* data of size n



How well it does



Q estimates explode!

How well it thinks it does

Kumar et al. "Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction" NeurIPS 2019

How to mitigate overestimation in offline RL?

This is the core goal of offline RL!

1. Constrain $\mathbf{a}'$ to stay close to behavior policy

- **Problem**: Q-function is unreliable on OOD actions
- **Solution**: Keep π close to π_β

π_β is <u>unknown</u> → Fit π to the data!

Policy update of TD3: $\pi = \arg\max_{\pi} \mathbb{E}_{\mathbf{s}\sim\mathcal{D}} \left[Q(\mathbf{s}, \pi(\mathbf{s})) \right]$

New policy update: $\pi = \arg\max_{\pi} \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim\mathcal{D}} \left[\lambda Q(\mathbf{s}, \pi(\mathbf{s})) - \overbrace{(\pi(\mathbf{s}) - \mathbf{a})^2}^{\text{BC}} \right]$

TD3+BC

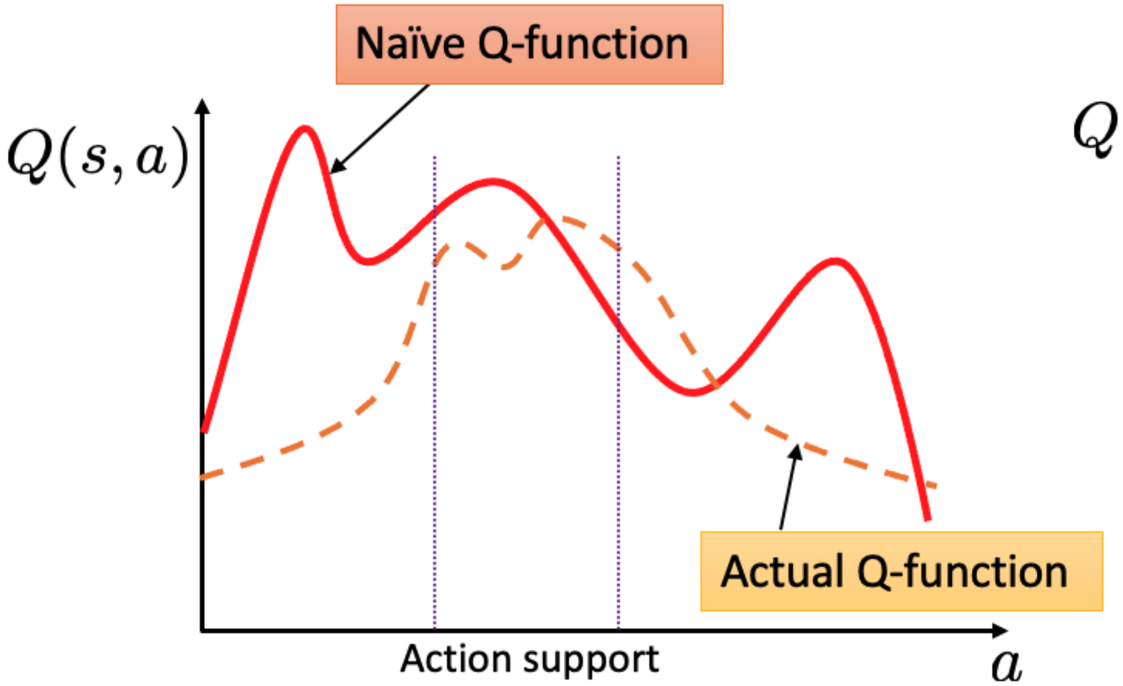
Fujimoto et al. "A Minimalist Approach to Offline Reinforcement Learning" NeurIPS 2021

2. Conservative Q estimation

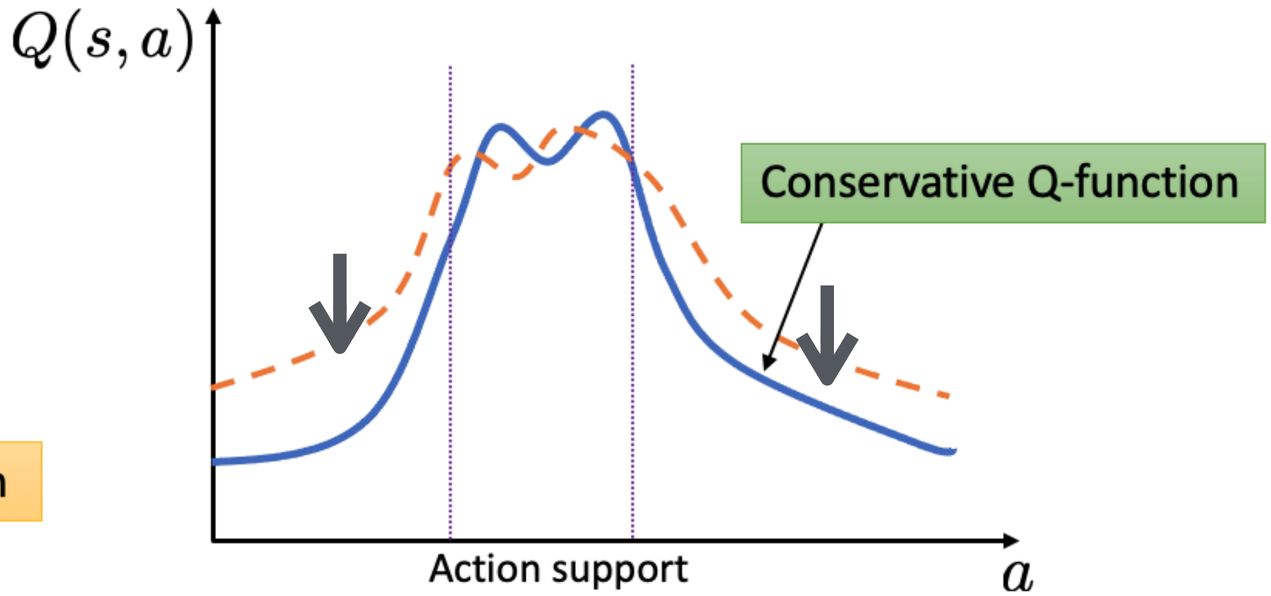
- Problems of policy constraint approaches (i.e. keep π close to π_β)
  - Too pessimistic (e.g. π_β is a random policy)
  - Not pessimistic enough (e.g. π can still deviate a little from data, incurring overestimation)
- Solution:
  - Option 1: Train Q so that OOD actions never have high values
  - Option 2: Avoid evaluating actions not in dataset

Conservative Q-Learning (CQL)

Learning conservative Q-functions



Naive Q-learning leads to overestimation of unseen actions



Let's underestimating Q-value for unseen actions

Conservative Q-Learning (CQL)

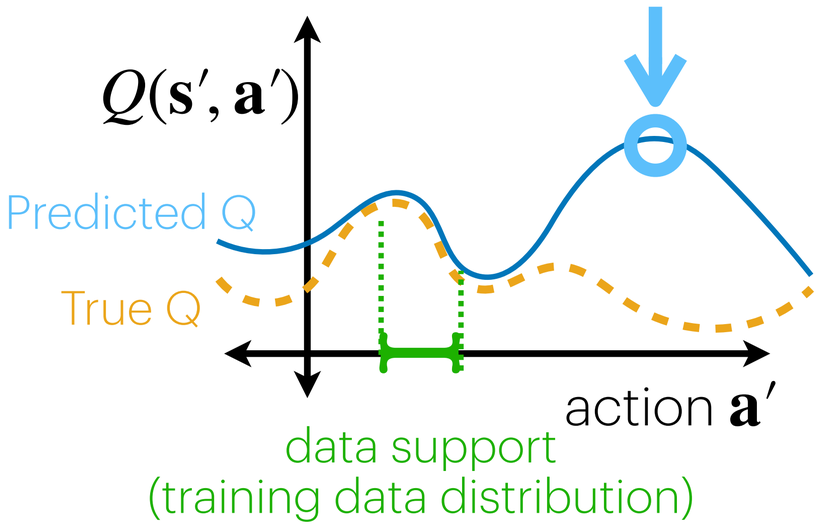
$$\hat{Q}^\pi = \arg\min_Q \max_\mu \mathbb{E}_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q(\mathbf{s},\mathbf{a}) - \left(r(\mathbf{s},\mathbf{a}) + \gamma\mathbb{E}_\pi[Q(\mathbf{s}',\mathbf{a}')]\right)\right)^2\right] + \alpha\mathbb{E}_{\mathbf{s}\sim D,\mathbf{a}\sim\mu(\cdot|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})]$$

Standard critic update

Push down large Q-values

Can show that $\hat{Q}^\pi \leq Q^\pi$ for large enough α

→ It might be too pessimistic. Can we make this bound tighter?



Conservative Q-Learning (CQL)

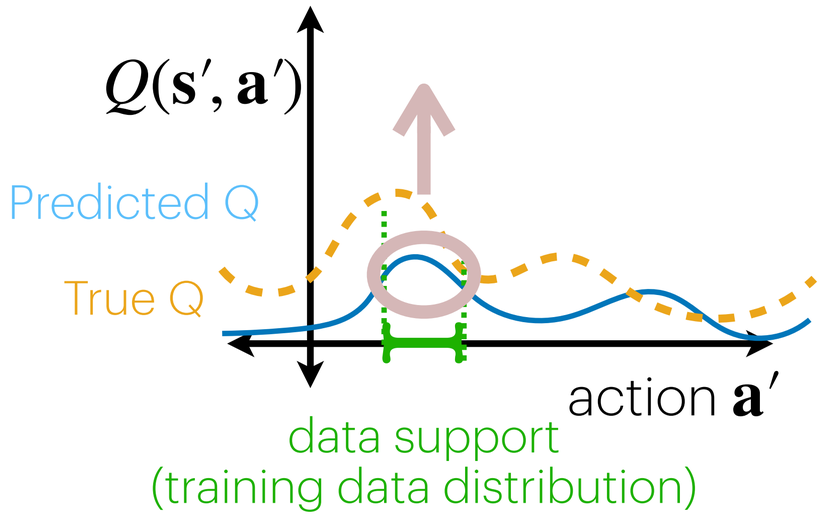
$$\hat{Q}^\pi = \arg\min_Q \max_\mu \mathbb{E}_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q(\mathbf{s},\mathbf{a}) - \left(r(\mathbf{s},\mathbf{a}) + \gamma\mathbb{E}_\pi[Q(\mathbf{s}',\mathbf{a}')]\right)\right)^2\right] + \alpha\mathbb{E}_{\mathbf{s}\sim D,\mathbf{a}\sim\mu(\cdot|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})]$$

Standard critic update

Push down large Q-values

$$- \alpha\mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}[Q(\mathbf{s},\mathbf{a})]$$

Push up Q-values for $(\mathbf{s},\mathbf{a})$ in data



No longer guaranteed that $\hat{Q}^\pi \leq Q^\pi$ for all $(\mathbf{s},\mathbf{a})$

But, guaranteed that $\mathbb{E}[\hat{Q}^\pi(\mathbf{s},\mathbf{a})] \leq \mathbb{E}[Q^\pi(\mathbf{s},\mathbf{a})]$ for all $\mathbf{s} \in D$

Conservative Q-Learning (CQL)

$$\hat{Q}^{\pi} = \arg\min_{Q} \max_{\mu} \mathbb{E}_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q(\mathbf{s},\mathbf{a}) - \left(r(\mathbf{s},\mathbf{a}) + \gamma \mathbb{E}_{\pi}[Q(\mathbf{s}',\mathbf{a}')]\right)\right)^2\right] + \alpha \mathbb{E}_{\mathbf{s}\sim D,\mathbf{a}\sim\mu(\cdot|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})]$$

$$- \alpha \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}[Q(\mathbf{s},\mathbf{a})]$$

How do we find μ and compute this term?

→ Adding regularizer $R(\mu) = \mathbb{E}_{\mathbf{s}\sim D}[H(\mu(\cdot\,|\,\mathbf{s}))]$ provides a closed form solution for μ

$$+ R(\mu)$$

Regularizer

→ With max entropy regularizer, optimal $\mu(\mathbf{a}\,|\,\mathbf{s}) \propto \exp(Q(\mathbf{s},\mathbf{a}))$

→ $\mathbb{E}_{\mathbf{s}\sim D,\mathbf{a}\sim\mu(\cdot|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})] + R(\mu) = \mathbb{E}_{\mathbf{s}\sim D} \log \Sigma_{\mathbf{a}} \exp(Q(\mathbf{s},\mathbf{a}))$

- In discrete action space, we can compute $\log \Sigma_{\mathbf{a}} \exp(Q(\mathbf{s},\mathbf{a}))$
- In continuous action space, we sample ~10 actions to approximate it

Conservative Q-Learning (CQL)

L_{CQL}

$$\hat{Q}^{\pi} = \arg\min_{Q} \mathbb{E}_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q(\mathbf{s},\mathbf{a}) - \left(r(\mathbf{s},\mathbf{a}) + \gamma\mathbb{E}_{\pi}[Q(\mathbf{s}',\mathbf{a}')]\right)\right)^2\right] + \alpha \log \Sigma_{\mathbf{a}} \exp(Q(\mathbf{s},\mathbf{a})) - \alpha\mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}[Q(\mathbf{s},\mathbf{a})]$$

CQL

- Update $\hat{Q}^{\pi}$ using L_{CQL} using D
- Update policy π using SAC

\+ Simple idea

\+ Work well in practice

\- Tuning α is crucial

\- A few hacks required (BC pre-training, estimating log-sum-exp)

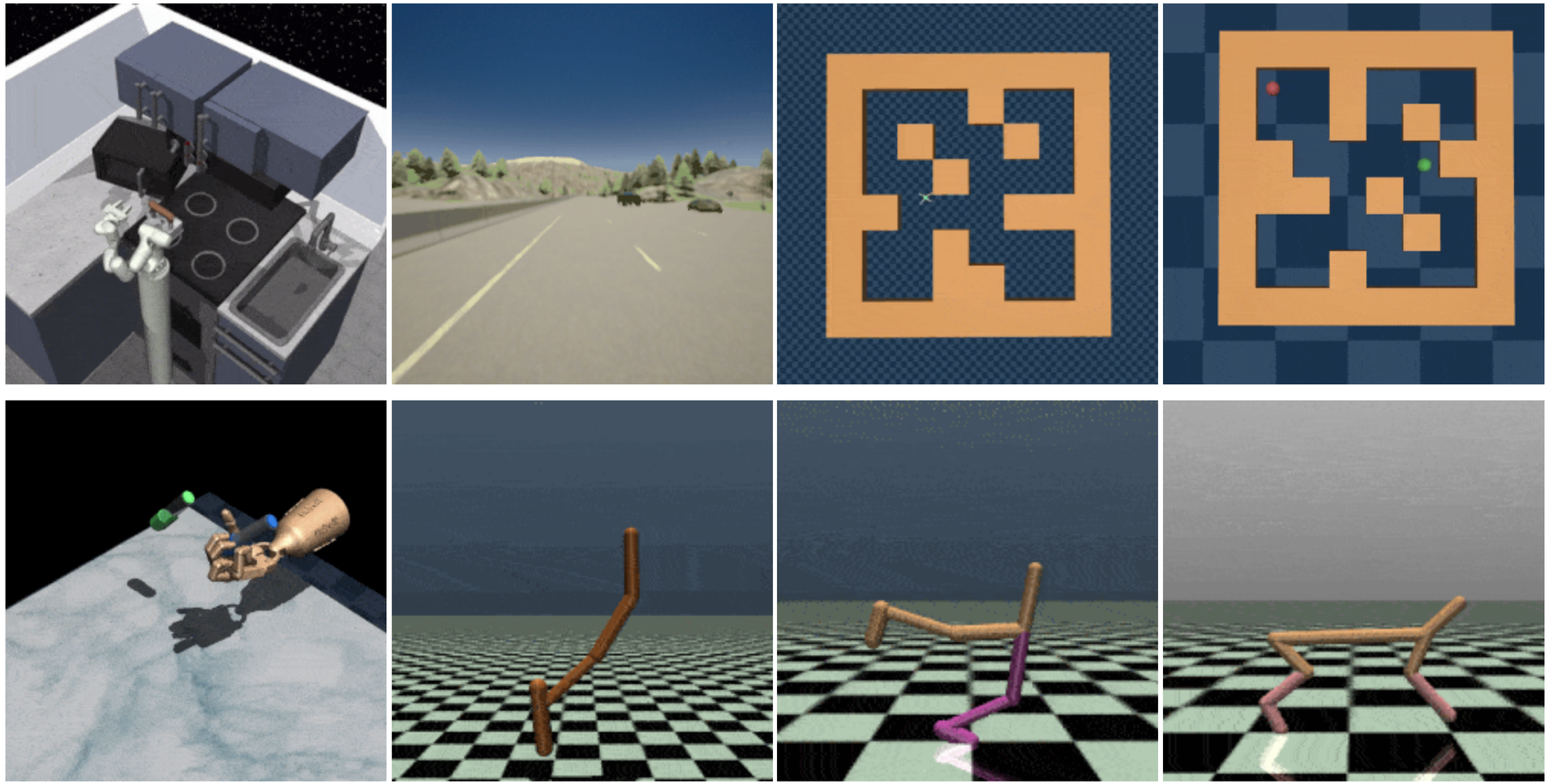
Simply replacing L_Q with L_{CQL} in SAC

You will implement CQL in HW5!

How to evaluate offline RL?

How do we evaluate offline RL methods?

- D4RL: Datasets for Deep Data-Driven RL
  - MuJoCo tasks
  - 1M samples
  - “random”
  - “medium-replay”
  - “medium-expert”



Fu et al. “D4RL: Datasets for Deep Data-Driven Reinforcement Learning” arXiv 2020

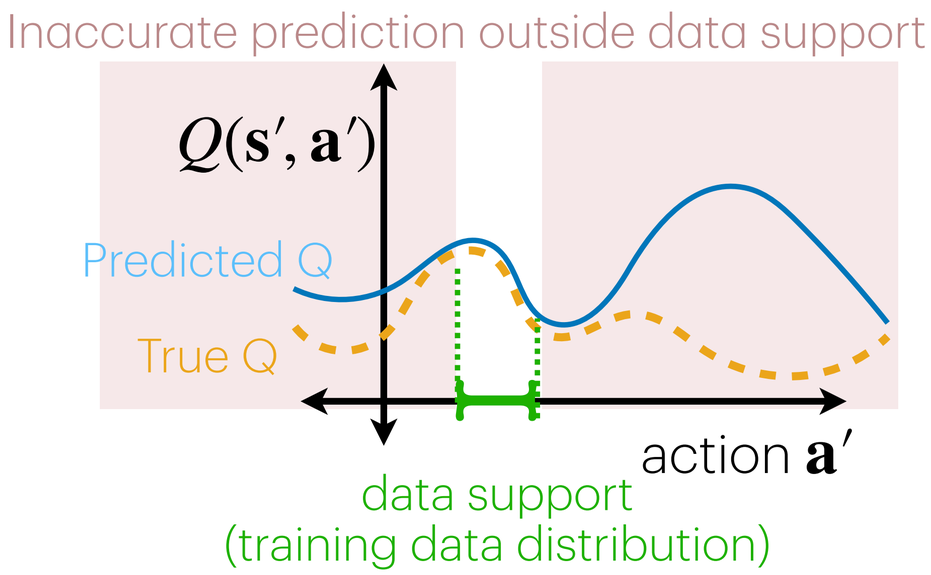
Dataset	BC	TD3+BC	CQL
halfcheetah-medium-v2	42.6	**48.3**	44.0
hopper-medium-v2	52.9	59.3	58.5
walker2d-medium-v2	75.3	83.7	72.5
halfcheetah-medium-replay-v2	36.6	**44.6**	**45.5**
hopper-medium-replay-v2	18.1	60.9	**95.0**
walker2d-medium-replay-v2	26.0	**81.8**	77.2
halfcheetah-medium-expert-v2	55.2	**90.7**	**91.6**
hopper-medium-expert-v2	52.5	98.0	**105.4**
walker2d-medium-expert-v2	**107.5**	**110.1**	**108.8**
locomotion-v2 total	466.7	**677.4**	698.5
antmaze-umaze-v0	54.6	78.6	74.0
antmaze-umaze-diverse-v0	45.6	71.4	**84.0**
antmaze-medium-play-v0	0.0	10.6	61.2
antmaze-medium-diverse-v0	0.0	3.0	53.7
antmaze-large-play-v0	0.0	0.2	15.8
antmaze-large-diverse-v0	0.0	0.0	14.9
antmaze-v0 total	100.2	163.8	303.6
total	566.9	841.2	1002.1
kitchen-v0 total	**154.5**	-	144.6
adroit-v0 total	104.5	-	93.6
total+kitchen+adroit	825.9	-	1240.3
runtime	10m	20m	80m

Summary of TD3+BC and CQL

- Explicitly constrain to the data (TD3+BC)
  - \+ intuitive
  - − often too conservative
- Implicitly constrain to data by penalizing Q-values (CQL)
  - \+ simple and work well in practice
  - − need to tune α

Summary of offline RL

- What is offline RL (also called batch RL)?
  - Learn a policy from a static (offline) dataset
- Why offline RL?
  - Online data is expensive. Reusing offline data is always good!
- Why is offline RL difficult?
  - Need to handle unseen actions in a safe way, while doing better than data
- Key challenges in offline RL
  - **Overestimating Q-values** due to shift between π_β and π_θ



Q-function is still unreliable on OOD actions

Q-function is unreliable on OOD actions

- **TD3+BC:** $\pi = \arg\max_{\pi} \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}\left[\lambda\, Q(\mathbf{s}, \pi(\mathbf{s})) - (\pi(\mathbf{s}) - \mathbf{a})^2\right]$

- **CQL:**

$$\hat{Q}^{\pi} = \arg\min_{Q} \max_{\mu} \mathbb{E}_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q(\mathbf{s},\mathbf{a}) - \left(r(\mathbf{s},\mathbf{a}) + \gamma \mathbb{E}_{\pi}[Q(\mathbf{s}',\mathbf{a}')]\right)\right)^2\right] + \alpha \mathbb{E}_{\mathbf{s}\sim D, \mathbf{a}\sim\mu(\cdot|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})] - \alpha \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}[Q(\mathbf{s},\mathbf{a})]$$

- We still use value estimates on **OOD actions**, which are likely **inaccurate**

Can we never estimate values on OOD actions?

Fujimoto et al. "A Minimalist Approach to Offline Reinforcement Learning" NeurIPS 2021

Leveraging reward in behavioral cloning

Behavioral cloning with rewards

- **Problem of BC**: Treat good and bad trajectories equally
- **Solution**: Imitate *only good* trajectories

Filtered Behavioral Cloning (K% BC)

1. Rank trajectories by return, $r(\tau) = \Sigma_t r(\mathbf{s}_t, \mathbf{a}_t)$
2. Filter dataset to include top K% of data: $\tilde{D} = \{\tau \mid r(\tau) > \eta\}$
3. Imitate filtered dataset: $\arg\max_{\pi} \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim\tilde{D}} \left[\log \pi(\mathbf{a} \mid \mathbf{s})\right]$

Behavioral cloning with advantage

- **Problem of Filtered BC**:
  - Treat transitions in a trajectory equally
  - Use reward only in a binary way (good or bad)

Advantage: $A(\mathbf{s}_t, \mathbf{a}_t) = Q(\mathbf{s}_t, \mathbf{a}_t) - V(\mathbf{s}_t)$

- **Solution**: Weight *each transition* depending on *how good the action is*!

Advantage-Weighted Regression (AWR)

1. **Fit $\hat{V}^{\pi_\beta}$:** $\arg\min_V \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[(R_t(\tau) - V(\mathbf{s}_t))^2\right]$

2. **Train $\hat{\pi}$:** $\arg\max_\pi \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[\log \pi(\mathbf{a}_t \mid \mathbf{s}_t)\exp\left(\beta(\underbrace{R_t(\tau) - \hat{V}^{\pi_\beta}(\mathbf{s}_t)}_{\text{Advantage: } \hat{A}(\mathbf{s}_t, \mathbf{a}_t)})\right)\right]$

Peng et al. "Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning" arXiv 2019

Behavioral cloning with advantage

Advantage-Weighted Regression (AWR)

1. **Fit $\hat{V}^{\pi_\beta}$:** $\arg\min_V \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[(R_t(\tau) - V(\mathbf{s}_t))^2\right]$
2. **Train $\hat{\pi}$:** $\arg\max_\pi \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[\log \pi(\mathbf{a}_t \mid \mathbf{s}_t)\exp\left(\beta(R_t(\tau) - \hat{V}^{\pi_\beta}(\mathbf{s}_t))\right)\right]$

Aside: Can show that advantage-weighted objective approximates KL-constrained objective.

$$\pi_{new} = \arg\max_\pi E_{\mathbf{a}\sim\pi(\cdot|\mathbf{s})} Q(\mathbf{s},\mathbf{a}) \text{ s.t. } D_{KL}(\pi \| \pi_\beta) < \epsilon$$

See Peters et al. (REPS), Rawlik et al. ("psi-learning")

Peng et al. "Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning" arXiv 2019

Behavioral cloning with advantage

Advantage-Weighted Regression (AWR)

1. **Fit $\hat{V}^{\pi_\beta}$:** $\arg\min_V \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[(R_t(\tau) - V(\mathbf{s}_t))^2\right]$
2. **Train $\hat{\pi}$:** $\arg\max_\pi \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[\log \pi(\mathbf{a}_t \mid \mathbf{s}_t)\exp\left(\beta(R_t(\tau) - \hat{V}^{\pi_\beta}(\mathbf{s}_t))\right)\right]$

+ Simple
+ Avoids training on any OOD actions

– $\hat{A}^{\pi_\beta}$ assumes weaker policy than $\hat{A}^{\pi_\theta}$
– Monte Carlo estimation $R_t(\tau)$ is noisy

Can we get $\hat{A}^{\pi_\theta}$, which is better than $\hat{A}^{\pi_\beta}$?

Peng et al. "Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning" arXiv 2019

Q-learning without estimating OOD actions

Q-learning: $L_Q(\phi) = \mathbb{E}_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q_\phi(\mathbf{s},\mathbf{a}) - (r(\mathbf{s},\mathbf{a}) + \underline{\gamma\mathbb{E}_{\mathbf{a}'\sim\pi(\cdot|\mathbf{s}')}[Q_{\phi'}(\mathbf{s}',\mathbf{a}')]})\right)^2\right]$

Problem: evaluate OOD actions

Solution: use in-distribution $\mathbf{a}' \sim D$

$\rightarrow \gamma\mathbb{E}_{\mathbf{a}'\sim D}[Q_{\phi'}(\mathbf{s}',\mathbf{a}')]$

SARSA: $L_Q(\phi) = \mathbb{E}_{(\mathbf{s},\mathbf{a},r,\mathbf{s}',\mathbf{a}')\sim D}\left[\left(Q_\phi(\mathbf{s},\mathbf{a}) - (r + \gamma Q_{\phi'}(\mathbf{s}',\mathbf{a}'))\right)^2\right]$

SARSA-style objective can learn Q-values **without estimating OOD actions**

Kostrikov et al. "Offline Reinforcement Learning with Implicit Q-Learning" ICLR 2022

Q-learning without estimating OOD actions

SARSA: $L_Q(\phi) = \mathbb{E}_{(\mathbf{s},\mathbf{a},r,\mathbf{s}',\mathbf{a}')\sim D}\left[\left(Q_\phi(\mathbf{s},\mathbf{a}) - (r + \gamma Q_{\phi'}(\mathbf{s}',\mathbf{a}'))\right)^2\right]$

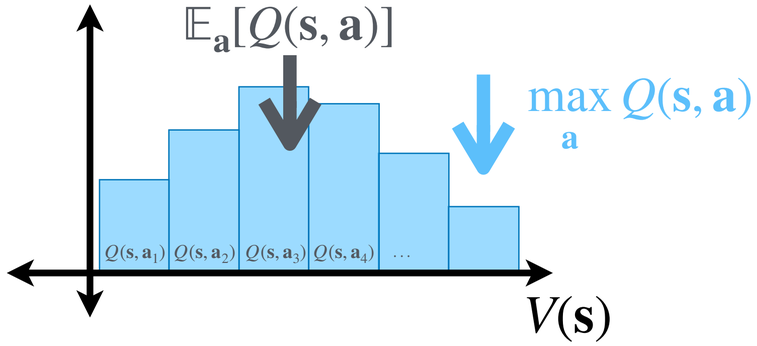
→ This uses both good and bad actions from π_β

π should use **only good actions** from π_β

$$L_Q(\phi) = \mathbb{E}_{(\mathbf{s},\mathbf{a},r,\mathbf{s}',\mathbf{a}')\sim D}\left[\left(Q_\phi(\mathbf{s},\mathbf{a}) - (r + \gamma \max_{\mathbf{a}'\in A \ s.t. \ \pi_\beta(\mathbf{a}'|\mathbf{s}')>0} Q_{\phi'}(\mathbf{s}',\mathbf{a}'))\right)^2\right]$$

→ The best sample of $V^{\pi_\beta}(\mathbf{s}') = \mathbb{E}_{\mathbf{a}'\sim\pi_\beta(\cdot|\mathbf{s}')}[Q(\mathbf{s}',\mathbf{a}')]$

Histogram of $V(\mathbf{s})$

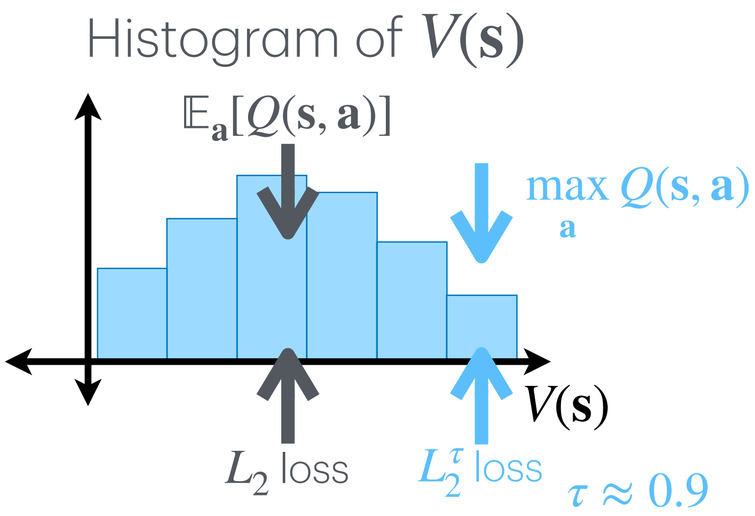


Kostrikov et al. "Offline Reinforcement Learning with Implicit Q-Learning" ICLR 2022

Value function with best actions

Typical state-value learning:

$$L_V(\psi) = \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}\left[\left(Q_{\phi'}(\mathbf{s},\mathbf{a}) - V_\psi(\mathbf{s})\right)^2\right]$$
$$= \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}\left[L_2\left(Q_{\phi'}(\mathbf{s},\mathbf{a}) - V_\psi(\mathbf{s})\right)\right]$$



Here, we use "expectile regression":

$$L_V(\psi) = \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}\left[L_2^\tau\left(Q_{\phi'}(\mathbf{s},\mathbf{a}) - V_\psi(\mathbf{s})\right)\right]$$

- $\tau = 0.9$ roughly means that $V_\psi(\mathbf{s})$ fits top-10% of $Q_\phi(\mathbf{s}, \cdot)$
- $\tau = 0.5$ roughly means that $V_\psi(\mathbf{s})$ fits mean of $Q_\phi(\mathbf{s}, \cdot)$
- $\tau = 0.1$ roughly means that $V_\psi(\mathbf{s})$ fits top-90% of $Q_\phi(\mathbf{s}, \cdot)$

Kostrikov et al. "Offline Reinforcement Learning with Implicit Q-Learning" ICLR 2022

Expectile regression

Expectile regression loss

x = target - prediction

$$L_2^\tau(x) = \begin{cases} (1-\tau)x^2 & \text{if } x < 0 \\ \tau x^2 & \text{otherwise} \end{cases}$$

$\tau = 0.9$

$$L_2^{0.9}(x) = \begin{cases} 0.1x^2 & \text{if } x < 0 \\ 0.9x^2 & \text{otherwise} \end{cases}$$

prediction is **larger** than target → small loss → prediction stays **large**
prediction is **smaller** than target → large loss → prediction becomes **larger**

→ prediction tends to map higher targets

Kostrikov et al. "Offline Reinforcement Learning with Implicit Q-Learning" ICLR 2022

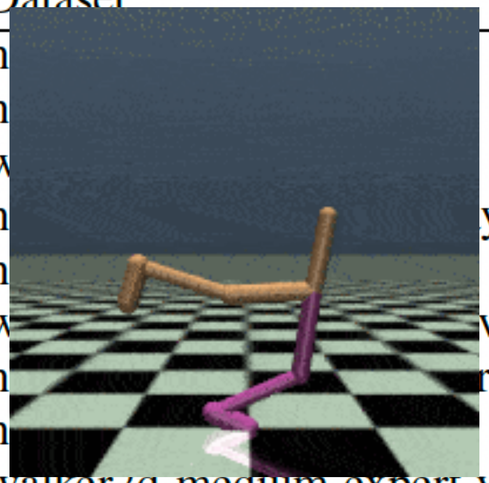
Implicit Q-Learning (IQL)

- Fit $V_\psi(\mathbf{s})$ with expectile loss on D: $\arg\min_\psi \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim D}\left[L_2^\tau\left(Q_{\phi'}(\mathbf{s},\mathbf{a}) - V_\psi(\mathbf{s})\right)\right]$
- Fit $Q_\phi(\mathbf{s},\mathbf{a})$ on D: $\arg\min_\phi \mathbb{E}_{(\mathbf{s},\mathbf{a},r,\mathbf{s}')\sim D}\left[\left((r + V_\psi(\mathbf{s}')) - Q_\phi(\mathbf{s},\mathbf{a})\right)^2\right]$
- Once converged, extract π using AWR (advantage-weighted regression)

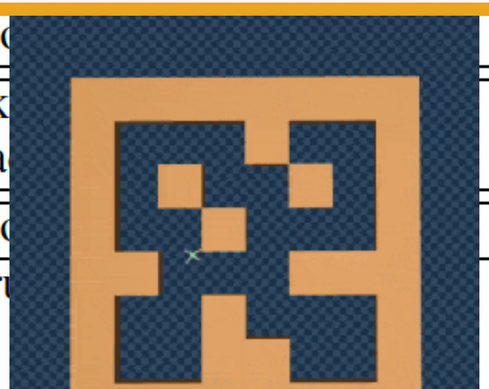
$$\arg\max_\theta \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim D}\left[\log \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)\exp\left(\beta(Q_\phi^{\pi_\theta}(\mathbf{s}_t,\mathbf{a}) - V_\psi^{\pi_\theta}(\mathbf{s}_t))\right)\right]$$

+ Avoids training on any OOD actions!
+ Policy (still) only trained on actions in data
+ Decoupling actor & critic training -> computationally fast
+ Work very well in practice
− Tuning τ and β is crucial

Kostrikov et al. "Offline Reinforcement Learning with Implicit Q-Learning" ICLR 2022

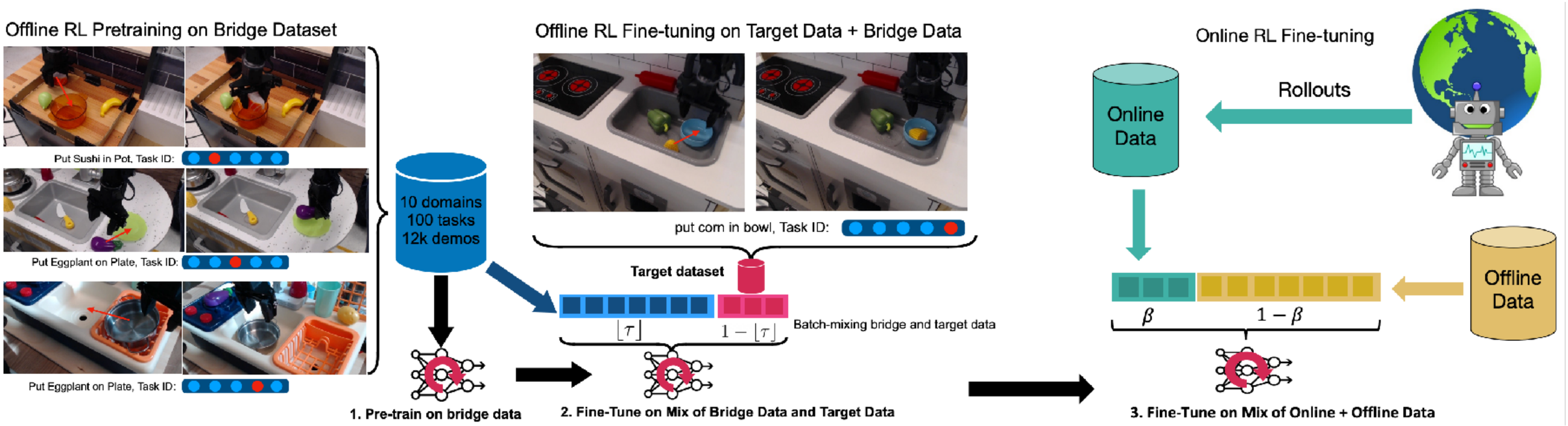


Dataset	BC	10%BC	DT	AWAC	Onestep RL	TD3+BC	CQL	IQL (Ours)
h	42.6	42.5	42.6	43.5	**48.4**	**48.3**	44.0	**47.4**
h	52.9	56.9	**67.6**	57.0	59.6	59.3	58.5	**66.3**
w	75.3	75.0	74.0	72.4	**81.8**	83.7	72.5	78.3
h y-v2	36.6	40.6	36.6	40.5	38.1	**44.6**	**45.5**	**44.2**
h	18.1	75.9	82.7	37.2	**97.5**	60.9	**95.0**	**94.7**
w v2	26.0	62.5	66.6	27.0	49.5	**81.8**	77.2	73.9
h rt-v2	55.2	**92.9**	86.8	42.8	**93.4**	**90.7**	**91.6**	86.7
h	52.5	**110.9**	**107.6**	55.8	103.3	98.0	**105.4**	91.5
walker2d-medium-expert-v2	**107.5**	**109.0**	**108.1**	74.5	**113.0**	**110.1**	**108.8**	**109.6**
locomotion-v2 total	466.7	**666.2**	**672.6**	450.7	**684.6**	677.4	698.5	692.4
antmaze-umaze-v0	54.6	62.8	59.2	56.7	64.3	78.6	74.0	**87.5**
antmaze-umaze-diverse-v0	45.6	50.2	53.0	49.3	60.7	71.4	**84.0**	62.2
antmaze-medium-play-v0	0.0	5.4	0.0	0.0	0.3	10.6	61.2	**71.2**
antmaze-medium-diverse-v0	0.0	9.8	0.0	0.7	0.0	3.0	53.7	**70.0**
antmaze-large-play-v0	0.0	0.0	0.0	0.0	0.0	0.2	15.8	**39.6**
antmaze-large-diverse-v0	0.0	6.0	0.0	1.0	0.0	0.0	14.9	**47.5**
antmaze-v0 total	100.2	134.2	112.2	107.7	125.3	163.8	303.6	378.0
t	566.9	800.4	784.8	558.4	809.9	841.2	1002.1	**1070.4**
k	**154.5**	-	-	-	-	-	144.6	**159.8**
a	104.5	-	-	-	-	-	93.6	**118.1**
t	825.9	-	-	-	-	-	1240.3	**1348.3**
r	10m	10m	960m	20m	$\approx$ 20m*	20m	80m	20m

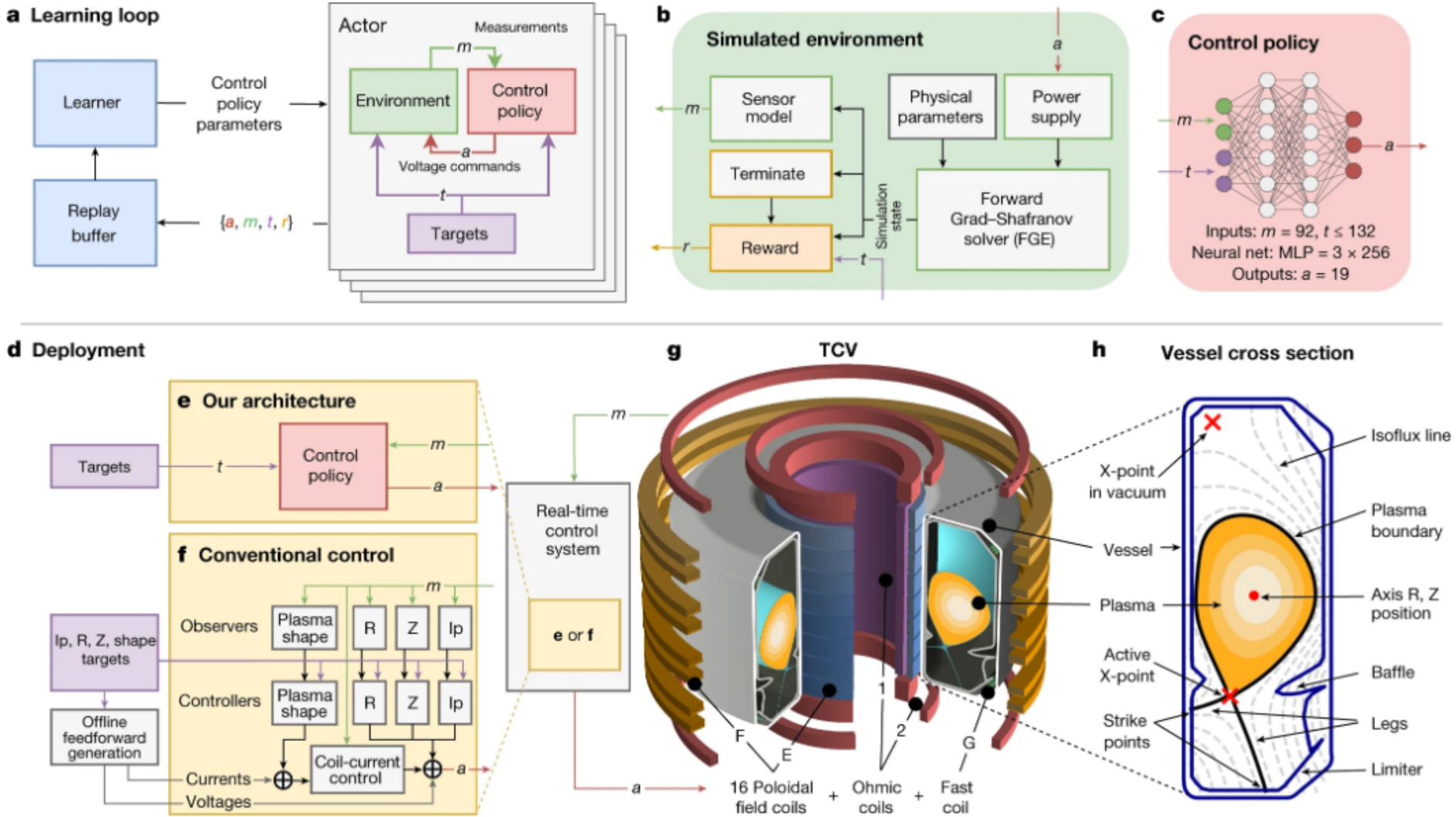


Where to use offline RL?

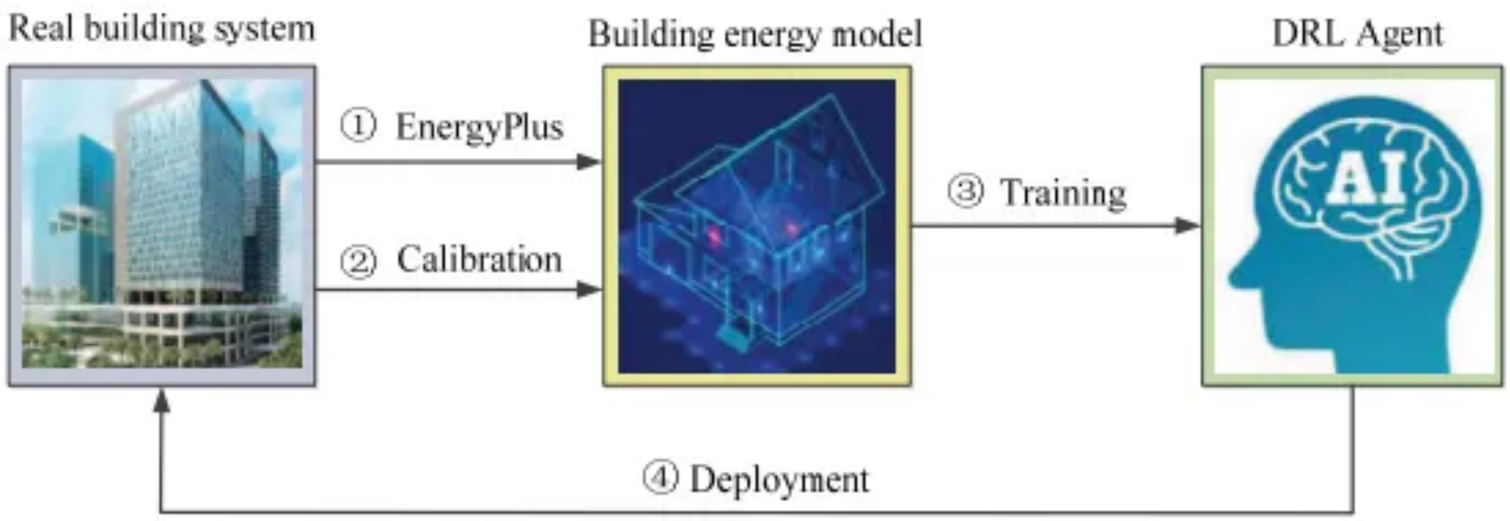
Pre-training for robots



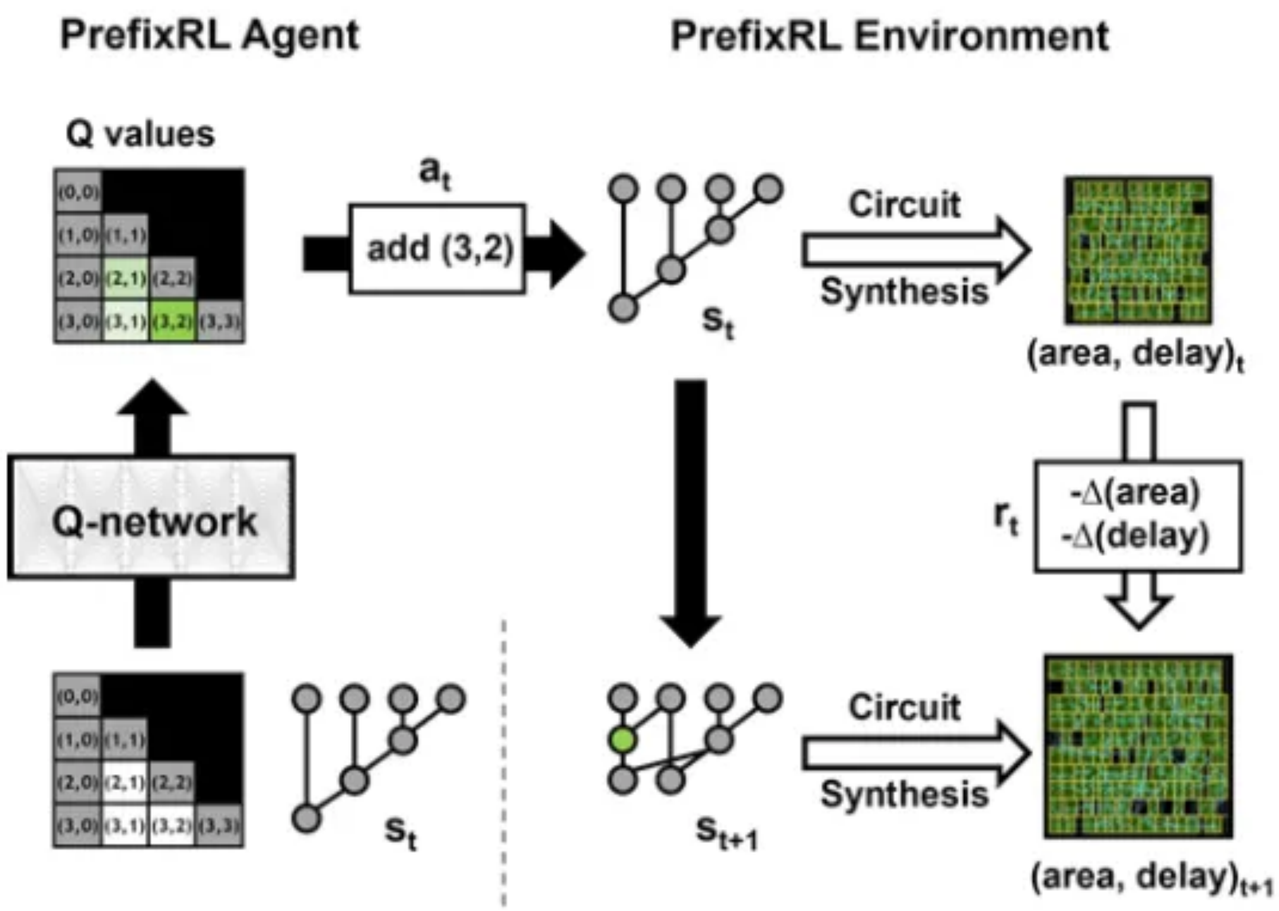
Nuclear fusion



Cooling system



Chip design


PrefixRL Agent
PrefixRL Environment
Q values
(0,0)
(1,0) (1,1)
(2,0) (2,1) (2,2)
(3,0) (3,1) (3,2) (3,3)
a_t
add (3,2)
s_t
Circuit
Synthesis
(area, delay)$_t$
Q-network
r_t
-Δ(area)
-Δ(delay)
s_t
s_{t+1}
Circuit
Synthesis
(area, delay)$_{t+1}$

Summary

- Offline RL: improving the policy beyond the *behavior policy*
  - Improving the policy requires estimating values for actions unseen in dataset
  - Policy improvement vs. distribution shift (inaccurate value estimate on OOD actions)
- Solutions:
  - TD3+BC: constraining the policy to limit how far it deviates from the behavior policy
  - CQL: regularizing the value functions to assign low values to OOD actions
  - Filtered BC: imitating only good trajectories
  - AWR: imitating only good transitions based on how good the actions are
  - IQL: never estimating values for OOD actions

Next class

- Model-based RL